This full-text paper was peer-reviewed and accepted to be presented at the IEEE ICCSP 2015 conference.

Design of Radix 4 Divider Circuit Using SRT Algorithm

Lohita S. Niwal, S. P. Hajare

Abstract—**The arithmetic operations are widely used in calculators and digital system .High speed methods of calculating are currently being requested, hence the design of fast divider is an important issues in high speed computing. In this paper we present fast radix-4 SRT division architecture with the digit-recurrent approach in which the quotient is obtained one digit per iteration. In this we estimating quotient digit instead of finding the exact one. The speculated quotient digit is used to calculate two possible partial remainders, in parallel with updating the new partial remainder for the next step whiles the quotient digit is being corrected. The two step processes does not affect the division speed, the approach has fast speed performance due to significant reduction in table size and by using higher radix .proposed divider takes power of 32.92µw with delay of 1.18 ns with 0.18µm CMOS technology.**

Index Terms—**CLA, CSA, SRT, Division**

I. Introduction

Many microprocessor and DSP chip perform mathematical operation such as multiplication, division subtraction, addition in hardware. However division is not carried out as fast as of other three. In high speed computation the design of fast divider is an important issue, and hence several mathematical algorithms have been developed till now to perform division rapidly, but very few of them are suitable for implementation in VLSI.

Division algorithms can be divided into five classes -digit recurrence, functional iteration, very high radix, look up table, variable latency [1]. Many practical division algorithms are hybrids of several of these classes [2]. Functional and digit recurrence are two main types of division algorithms. Functional algorithm is base on multiplication and Digit recurrence take advantage of subtraction. Newton Raphson and Goldschmidt are example of functional algorithms and they are under fast division algorithm because they produced twice as many digits of the final quotient in each iteration. Digit recurrence algorithm is slow division algorithm and they produce one digit of the final quotient per iteration.SRT is very commonly use as digit recurrence. The speed of SRT-based divider is mainly determined by the complexity of the quotient-digit selection. The use of QST (quotient select table) significantly reduces the complexity of quotient-digit selection [3]. However, the table size increases drastically with high radices [4]. The table size can be reduced significantly by estimating the quotient digit instead of finding the exact one [5]. Radix 4 is typically used for SRT divider implementation with this 2 quotient bit are retired with each iteration of the algorithm, therefore the SRT algorithm takes n/2 iteration for an n bits result. In this paper in first section we present higher radix SRT division algorithm and in second we present propose SRT radix-4 divider and in third section we present the implementation of proposed Divider.

Lohita.S.Niwal is student of M.Tech Electronics Engineerring(VLSI Design), Yeshwantrao Chavan College of Engineering Nagpur, India (email: lohita.n80@gmail.com)

S.P.Hajare is Asst prof with the Department of Electronics Engineering, Yeshwantrao Chavan College of Engineering Nagpur, India (email: shwetahajare29@gmail.com).

II. SRT Division Algorithm

SRT is name after Sweeney Robertson and Tocher independently develop division algorithm nearly at same time. Atkins provides a good analysis of SRT algorithm [6] while Tan present higher radix SRT Division and it first introduced by Robertson.

A. *High-Radix SRT Division*

The partial Remainder in a high-radix SRT division follows the recursive equation,

$$r_i = \beta r_{i-1} - q_i D \qquad (1)$$

Where $\beta=2^m$ is the radix, D is the divisor, r_{i-1} is the partial remainder at the $(i-1)^{th}$ step and q_i is the i^{th} quotients digit. For N-bit division, where N=2n ,it takes (n-m)iterations to compute the division process The division process needs a table QST (quotients select table) to determine the quotients q_i and the remaining components compute eq. 1.The high-radix SRT division can be implemented in such a way that its quotient digit q_i is selected From a digit set $\{-\alpha\ldots 0, 1 \ldots \alpha\}$ where $[(\beta-1)/2] \leq \alpha \leq (\beta-1)$ The ratio $k= \alpha / (\beta-1)$ is a measure of the redundancy in the representation of the quotient digits. The smaller the value of k, the smaller is the redundancy in the number system for the quotient.fig.1(a) show higher radix SRT division with quotient digit selection Table(QST).and fig.1(b) is radix 4 SRT division with QST For radix 4 division $(\beta,\alpha)=(4,2)$, $k= \alpha / (\beta-1)=2/3$ and $(D_{min}, D_{max})=(0.5,1.0)$.The quotient digit q {-2,-1,0,1,2}Let P=4ri-1 denote the

978-1-4799-8081-9/15/$31.00 © 2015 IEEE

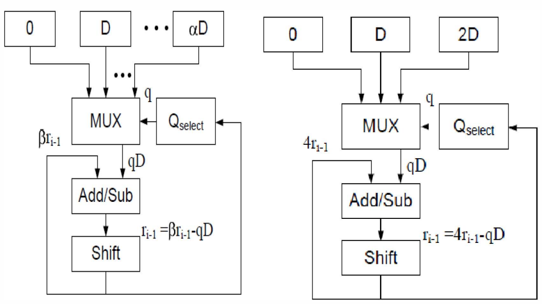

Fig. 1. (a) Higher Radix SRT Division (b) Radix 4 SRT Division[2]

previous partial remainder, where r_i-1 is the partial remainder at the $(i\text{-}1)^{th}$ step and $|r_i\text{-}1| \leq (2/3)D$. Thus, the upper limits and lower limits [7] for P are

$$(-2/3+q)D \leq P \leq (2/3+q)\, D \tag{2}$$

B. *Proposed Radix 4 SRT Divider*

In order to reduce the hardware cost of building QST (Quotient selection table) an efficient SRT division with QST was proposed illustrated in fig.2. Instead of selecting the correct quotient digit q ,$-\alpha \leq q \leq \alpha$,the approach first estimates a quotient digit $q^{\#}$, $-\alpha \leq q^{\#} \leq \alpha$-1,such that $\{q^{\#},q^{\#}+1\}$, in other word the actual quotient digit is either $q^{\#}$ or $q^{\#}$+1,and the possible partial remainder is either $P^0 = 4r_{i-1} - q^{\#}D$ or $P = 4r_{i-1} - [-(q^{\#}+1)D]$,thus the actual quotient digit q=q#+q*,where q* is correction value is either 0 or 1 [2].For estimated quotient digit $q^{\#}$, the upper and lower limits for the corresponding partial remainder P are

$$(-2/3+q^{\#})\, D \leq P \leq (2/3+q^{\#}+1)\, D \tag{3}$$

Table I lists all possible region for various $q^{\#}$ thus the overlapping are listed in table II and plotted in fig .1 where p is represented in two's implement form. From overlapping region three comparisons constant is generated they are C_1= (000.1), C_0=0 and C_2= (111.1) thus quotient digit is estimated by [8]

$$q^{\#} = \begin{cases} 1 \; if \; C_1 \leq P \\ 0 \;\; if \; C_0 \leq P \leq C1 \\ -1 \;\;\; if \; C_{-1} \leq P \leq C_0 \\ -2 \; if \; P \leq C_{-1} \end{cases} \tag{4}$$

TABLE I
REGION OF PARTIAL REMAINDER FOR ESTIMATED QUOTIENT DIGIT

$q^{\#}$	Partial remainder P
1	1/3 D ≤ P ≤ (8/3)D
0	-2/3 D ≤ P ≤ (5/3)D
-2	-8/3 D ≤ P ≤ (-1/3)D
-1	-5/3 D ≤ P ≤ (2/3)D

TABLE II
OVERLAPPING REGION FOR ESTIMATED QUOTIENT DIGIT

	Partial remainder P
P0,1	1/3 D ≤ P ≤ (5/3)D
P-1,0	-2/3 D ≤ P ≤ (2/3)D
P-2,-1	-5/3 D ≤ P ≤ (-1/3)D

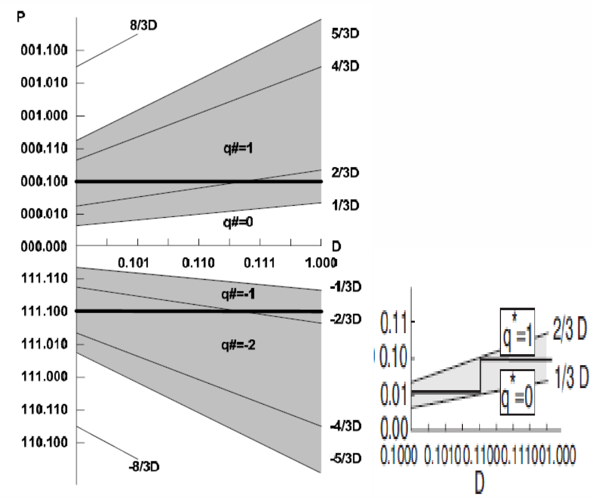

Fig. 2. Estimated quotient digit selection [2]

After estimating the quotient digit $q^{\#}$, the actual quotient digit q is either q# or $q^{\#}$+1.the range of partial remainder is [9]-[10].

$$(-2/3D) \leq P^* = P\text{-}\, q^{\#}D \leq= (5/3)\, D \tag{5}$$

And q* is determined by using comparison constant C_2=0.01 for D [0.10, 0.11) and C_3= (0.10) for D= [0.11, 1.00) as shown in Table II. Once q* is determined the actual $q=q^{\#}+q^*$ and the partial Remainder is P^0 if q*=0 or P^1 if q*=1

$$q^* = \begin{cases} 1 \;\; if \; C_2 \leq p^* \; and \; D = [0.10_2, 0.11_2) \\ \qquad\qquad or \\ \;\; if \; C_3 \leq P^* \; and \; D = [0.11_2, 1.00_2) \\ 0 \;\; Otherwise \end{cases} \tag{6}$$

Or

$$q^* = \begin{cases} 1, if \; (C_2 \leq p^*D < 0.11) \\ \qquad\qquad or \\ \qquad\quad C_3 \geq P^* \end{cases} \tag{7}$$

III. IMPLEMENTATION OF SRT DIVIDER

The radix 4 SRT divider is shown in fig.3 this design consists of the divisor selector i.e. MUXC block partial remainder generator, QS, QH, QC and MUX Block.

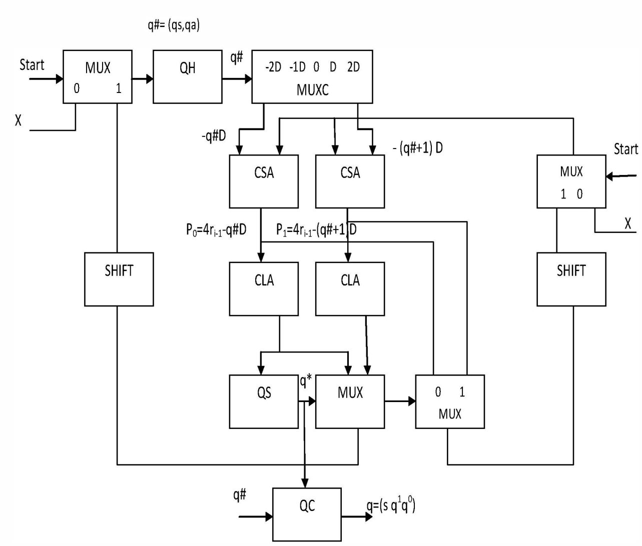

Fig. 3. Divider Circuit

According to Algorithm, QH block estimate the quotients $q^{\#}$, and the Divisor selector MUXC takes the estimated digit $q^{\#}$ to generates both $-q^{\#}D$ and $-(q^{\#}+1)D$. the partial remainder generator realized by two CSA (carry Save Adder) compute Both $P^0 = 4r_{i-1} - q^{\#}D$ and $P^1 = 4r_{i-1} - [-(q^{\#}+1)D]$.The QS table generates q* to determined either p0 or p1 is selected as partial remainder for next cycle. Both q^* and $q^{\#}$ generates the final quotients digit by QC the first seven bits of the partial remainder are then shifted and determined $q^{\#}$ in QH table.It is assume that the divisor D is n bit positive normalized binary number, where D= (0.1 d-2 d-3 ...d-n) ,the dividend X is k bit binary ranged between –(8/3)and (8/3),where X is represented in two's compliment form and k=2n.and quotient q is n bit form {-2,-1,0,1,2}[2].in the first cycles the first n+3 bits of X are processed and in next following cycle two new bits of X are shifted into the updated remainder[2] .

A. *QH Block*

We first estimate the quotient digit $q^{\#}$ from Block QH as shown in fig.4 (a) [2]. The multiplexer circuit takes the quotient digit $q^{\#}$ and generates $q^{\#}D$ and $-(q^{\#}+1)D$.Two adders are used to compute $P^0 = 4r_{i-1} + (-q^{\#}D)$ and $P^1 = 4r_{i-1} + [-(q^{\#}+1)]D$ and the results are compared with the constants to generate q* and $q^{\#}$. The correction value q* selects the updated remainder r_i from either P^0 or P^1, and the first Seven most significant bits of the updated remainder. The updated remainder is shifted left by 2 Bits to block QH for generating q#. Thus, after shifting 2 bits, we obtain (SP_{-1} P_{-2} P_{-3}) which is used to compare the constants given in Table III. According to the encoding scheme in Table V, eqn. 5 implies that qs. S and q^a. If (P_{-1} P_{-2} P_{-3}). (00.0) or (11.1), where q^a is expressed by the following logic function

$$q^a = [p_{-1}p_{-2}p_{-3} + p'_{-1}p'_{-2}p'_{-3}] \tag{8}$$

B. *QC Block*

QC block is shown in fig. 4(c) this block do addition of the estimated quotient digit $q^{\#}$ and correction value q^* to produce the actual i^{th} quotient digit $qi=q^{\#}+q^*$, where qi {-2,-1, 0, 1, 2} [2].q= ($S^q q^1 q^0$) be in sign Magnitude form, where sq is the sign bit and ($q_1 q_0$) represent the binary representation of the absolute value of q[]. Table IV show the truth table for QC

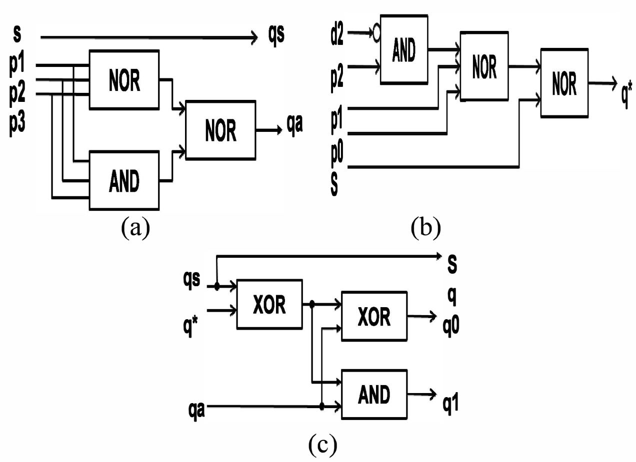

Fig. 4. (a) QH Block (b) QS Block (c) QC Block

TABLE III
COMPARISON CONSTANT FOR $Q^{\#}$

SP-1P-2P-3	q#	qs	qa
011.1	x	x	x
01x.x	1	0	1
001.x	1	0	1
000.1	1	0	1
000.0	0	0	0
111.1	-1	1	0
111.0	-2	1	1
110.x	-2	1	1
10x.x	-2	1	1
100.0	x	x	x

q=(S_qq1q0) in terms of q^s,q^a,q* S_q= q^s, q1= q^a(q^s XOR q*)and q^0= q^a XOR(q^s XOR q*)

C. *QS Block*

QS Block is shown in fig.2 (b) this block generates the correction value q* from the already computed remainder P^0 [2].and update partial remainder 4ri-1 and estimate quotient digit $q^{\#}$. q* is determined by the comparison constants. Let P_0= (SP_1 P_0 P_{-1} . . . P-n). Since the maximum positive value of P0 is 5/3 and the bit P1=0 Is true for all positive P0, the bit P1 is ignored here. The comparison constant can be tabulated as in table IV. For a P0, c3= (000.10) <=P0, it can be Represented by (SP_1 P0.P_{-1} . . . P-n) = (001.xx . . . x) or (000.1 x…x), and will be truncated as ($sp_0p_{-1}.p_{-n}$) = (001.x) or (000.1). bit p1 is always 0 for a positive p0.thus the stamens c3<=p_0 is equivalent to S=0 & {p_0=1 or p_{-1}=1}.similarly the statement C_2<=p is equivalent to S=0 & {P_0=1 or P_{-1}=1 or P_{-2}=1}.since D([0.5,1] a value D<0.11is express as D = (0.1d-2d-d_t) = (0.10xxxxx..x).Thus D < (0.11) is equivalent to d-2=0

$$q^* = 1\, if \begin{cases} \{S = 0\,\&[P_0 = 1\, or P_{-1} = 1]\} \\ \qquad or \\ \{S = 0\,\&[P_0 = 1\, or P_{-1} = 1\, or\, P_{-2=1}]\&d_{-2} = 0\} \end{cases} \tag{9}$$

And the logic function is …

q*= S' (P0+P -1) +S' (P_0+P_{-1}+P_{-2}) d_{-2}'

= S' (P_0+P_{-1}+$P_{-2}$$d_{-2}$'

TABLE IV
TRUTH TABLE OF QC

$q^{\#}$ q^s q^a	q*=0	q*=1
	q S^q q^1 q^0	q S^q q^1 q^0
1 0 1	1 0 0 1	2 0 1 0
0 0 0	0 0 0 0	1 0 0 1
-1 1 0	-1 1 0 1	-0 1 0 0
-2 1 1	-2 1 1 0	-1 1 0 1

TABLE I
COMPARISON CONSTANT FOR Q*

q*	d-2	
$SP_0P_{-1}P_{-2}$	0	1
01.XX	1	1
00.10	1	1
00.01	1	0
00.00	0	0
1X.XX	0	0

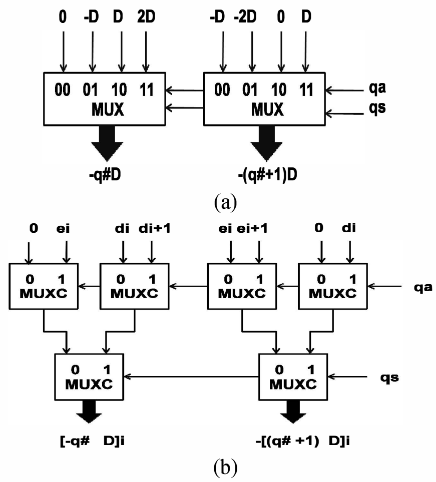

Fig. 5. MUXC Circuit

D. *MUXC Block*

Multiplexer circuit MUXC is shown in fig. 5 which choose input from {-2D,-1D, 0, 1D, 2D}and generate output $-q^{\#}D$ and - $(q^{\#}+1)$ which are choose from {-2D,-1D, 0, D, 2D}.divisor D is an n bit positive normalized binary number .number –D and -2D are represented in two's complement let D is denoted as $(S^{d}d_{o}.d_{-1}d_{-2}\ldots d_{-n})$, where form $S^{d}=0$ is sign bit d0=0 and $d_{-1}=1$.then –D = ((s-d)'e0.e-1e-2…e-n) and -2D= ((sd)'e-1e-2..e-n0), where $e_i=d_i$' and e_{-n}=d-n'+1.MUXC block diagram is shown on fig.5. each bit slice of the MUXC is realized by six 2:1 multiplexers (MUX) select input of first level MUX is q^{a} and second level MUXs is q^{s} .the output of i^{th} it slice in MUXC is fed to CSA to generate initial carry of CSA[] .truth table for MUXC is shown in Table VI

TABLE VI
FUNCTION MUXC AND BIT ASSIGNMENT FOR (-N) BIT OF CSA

$q^{\#}$	q^{s}	q^{a}	$-q^{\#}D$	y_{-n}	$-(q^{\#}+1)D$	y_{-n}
1	0	1	-D	1	-2D	1
0	0	0	0	0	-D	1
-1	1	0	D	0	0	0
-2	1	1	2D	0	D	0

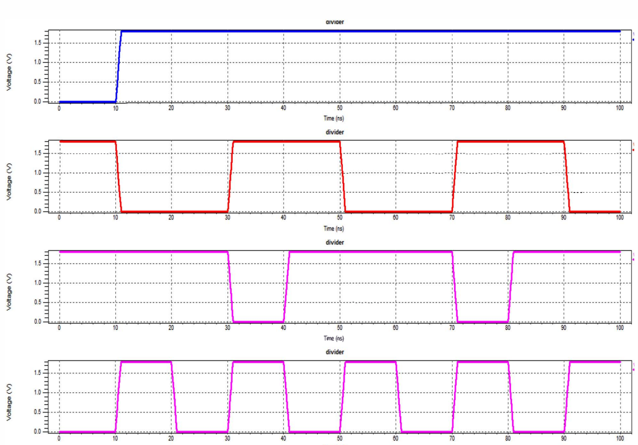

Fig. 6. Divider Circuit waveform

IV. CONCLUSION

Digit –recurrence division is an algorithm in which the quotient is obtained one digit per iteration .The algorithm have been extensively studied for the design trade off among delay and power .This paper developed the detailed implementation of the fast radix -4 division algorithm. The speculated digit is used to compute the two possible partial remainders, for the next step, in parallel with the quotients digit correction process. The focus was on a speed up of each iteration cycle and reduced block area, leading short division time. The proposed SRT algorithm is implemented with a delay of 1.18ns and average power is 32.92µw although this paper presented only for radix- 4 divisions the same design concept is readily extended for higher radices to reduced the quotient digit selection table size. And also similarly to this the proposed architecture we can also be implemented for square root calculation.

REFERENCES

[1] Stuart F. Oberman, Michael J. Flynn "Division Algorithms and Implementations" *IEEE Transactions On Computers, Vol. 46, No. 8, August 1997*

[2] C.-L.Wey and C.-P.Wang, "Design of a fast radix-4 SRT divider and its VLSI implementation", *Computers and Digital Techniques, IEE Proceedings, Jul 1999, Vol. 146, Issue. 4, p. 205-210*

[3] D.E.Atkins,high Radix Division using Estimates of the Divisor and partial Remainder .in international confreance on Computer design,1988.

[4] Mohamed Anane, Hamid Bessalah, Mohamed Issad, Nadjia Anane, and Hassen Salhi "Higher Radix and Redundancy Factor for Floating Point SRT Division" *IEEE Transactions on very large scale Integration (VLSI) systems, vol. 16, no. 6, June 2008*

[5] Nishant r srivastava"Radix 4 SRT division with quotient prediction and operand scaling"2007

[6] Tung N. Pham and Earl E.Swartzlander, Jr."Design of Radix-4 SRT Dividers in 65 Nanometer CMOS Technology"*IEEE Computer Society 2006*

[7] A. Nannarelli and T. Lang "Low-Power divider" IEEE Transactions On Computers, Vol. 48, No. 1, January 1999

[8] C.L. Wey, "Design of fast high-radix SRT dividers and their VLSI implementation."IEE Proceedings, Computers and Digital Techniques, vol.147, Issue 4, pp. 275-282, July 2000

[9] PAN, T.-H., KAY, H.-S., CHUN, Y., and WEY, C.L.: `High-radix SRT Division with speculation of quotient digits', Proc. International Conference of Computer Design (ICCD), Austin, Texas, 1995, pp. 479±482

[10] T.K. Fachhochschule, "Design of a configurable speed optimized Radix-4 Hardware Divider." Master Thesis, Carinthia Tech Institute University, July 2002.